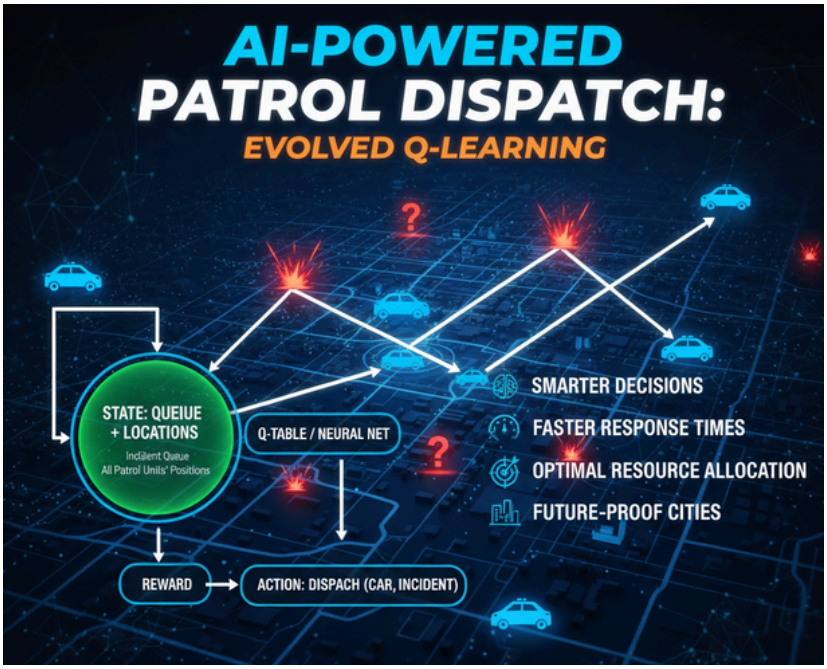
Department of Computer Science & Engineering

Project Title: Q-learning for patrol dispatch (state: queue+locations).



Student Name: Rahul Mehta
Student ID: 25SCS1003003792
Module: Introduction to AIML
Section: 1CSE29
Submitted To: Dr. Prakash Pal

AI-Powered Patrol Dispatch using Evolved Q-Learning

Abstract:

To ensure the safety of urban centers, the dispatching of emergency patrols needs to be done as quickly as possible. Many legacy systems apply rudimentary, rule-based logic, such as directing the nearest free unit, which in more complex scenarios is counterproductive. This solution proposes building an intelligent dispatching agent based on the Q-learning algorithm, which is a type of reinforcement learning. The innovation is in the complete state representation in which the agent is provided a panoramic view of the situation. This captures the entire queue of incidents and the real-time positions of all the patrol units. This allows the agent to learn complex, strategic policies in a simulated environment designed to optimize system-wide response times and resource utilization efficiency.

Introduction:

Assigning a small number of patrol units to a constantly changing stream of incidents, from minor disturbances to serious emergencies, is a constant challenge for dispatch operators in a busy city. Although minimizing response time is always the aim, the optimal choice is not always clear. A critical zone might not be protected if the only unit in the area is sent to a low-priority call. An artificial intelligence solution would be a perfect fit for this challenging optimization problem.

This project is to present a proof of concept AI dispatcher which we train via action in a simulated environment. We will use Q-learning to train an agent in which it makes dispatch decisions (Actions) out of what the present situation (State) is in for the best total score (Reward). We put forth a new state representation which the agent uses to think stratagically as our main contribution

Core Concept: Q-Learning Explained:

Q-learning is like teaching a robot to navigate a maze for a piece of cheese. The robot doesn't have a map; it just learns from its actions.

- State (S): Where the robot is in the maze.
- Action (A): A direction the robot can move (left, right, up, down).
- Reward (R): It gets a big positive reward (+100) for finding the cheese and small negative rewards (-1) for each step it takes (to encourage it to be fast).

The "brain" of the robot is a Q-table. This table stores a score (Q-value) for every possible action in every possible state. Initially, all scores are zero. As the robot explores, it updates these scores using the Bellman Equation:

$$Q_{new}(s,a) \leftarrow Q_{old}(s,a) + \alpha \cdot (R + \gamma \cdot \max_{a'} Q(s',a') - Q_{old}(s,a))$$

- $Q(s,a)$ is the Q-value for taking action a in state s.
- α (alpha) is the learning rate (how much we trust the new information).
- γ (gamma) is the discount factor (how much we value future rewards).
- R is the reward received.
- $\max_{a'} Q(s',a')$ is the best possible score from the next state s'.

Over thousands of trials, the robot fills its Q-table with values that represent the best path from any point in the maze. Our dispatch agent learns in the exact same way, but its "maze" is the complex world of city incidents.

Methodology:

A.Libraries Used:

- **NumPy:** Handles arrays and matrices efficiently, which is useful for managing the Q-table and performing calculations.
- **random:** Essential for creating random incidents in the environment and for implementing the epsilon-greedy policy for exploration.
- **itertools:** Used to efficiently create the action space for the agent, containing all possible car-incident dispatch combinations.

B. Environment & Model:

- **Simulated Grid Environment:** The project uses a custom-built grid world to simulate the city. Patrol cars move, and incidents appear randomly within this grid. The environment is responsible for calculating distances, updating locations, and providing rewards based on the agent's actions.
- **Q-Learning Model:** The core of the system is the Q-learning agent, which consists of a Q-table (implemented as a Python dictionary) and the logic for the Bellman equation to update its policy. The agent learns by interacting with the simulated environment over thousands of episodes.

Implementation:

1.Holistic State Representation (queue+locations):

The system creates a unique state by putting together the list of all active incidents (with their priorities and locations) and the current locations of all patrol cars. The agent can keep an eye on the whole operational landscape because it knows the centroid of the situation for each state. This broad view lets the agent make strategic decisions instead of just reacting to what happens.

2. Strategic Dispatch Logic:

The agent evaluates the situation and chooses an action (for instance, Send Car 1 to Incident 2). The agent uses the Q-table to make this choice. The Q-table contains the potential future reward associated with each state-action pair. Every time an object (incident) moves from the “unresolved” to “resolved” state, a reward is earned.

3. Reward Function Visualization:

Analyzing the agent's learning as well as the performance can be done by visualizing total reward each episode. An agent is learning an optimal dispatch policy if they see total reward as a consistently increasing curve. Conceptually, this is similar to a movement heat map, where brighter areas correspond to routes taken more often during the optimal policy and to actions that resulted in higher rewards, and thus "visited" more often.

Results and Discussion

After training for thousands of episodes, the q_table dictionary will be populated with states and their corresponding action values. This table now represents the learned intelligence of the agent. When presented with a real-world situation (a state), the agent can simply look up that state in its Q-table and choose the action with the highest associated Q-value.

The effectiveness of the agent can be measured by running it in a "test mode" (with exploration turned off, i.e., epsilon = 0) and comparing its performance metrics (like average response time per incident) against a baseline algorithm, such as a simple "always dispatch the closest car" strategy. It is expected that the Q-learning agent will demonstrate superior performance, especially in scenarios with multiple simultaneous incidents of varying priorities, showcasing its ability to make strategic sacrifices for a better overall outcome.

Future Scope and Enhancements:

This project serves as a base which may be expanded in many ways:

- **Deep Q-Networks (DQN):** For a more realistic simulation with a larger grid and more cars/incidents, the number of possible states becomes enormous. We can replace the Q-table with a neural network that approximates the Q-values. This is known as Deep Q-Learning.
- **Dynamic Factors:** The state representation can be enriched to include real-time traffic, time of day, or weather, making the agent even more context-aware.
- **Multi-Agent Reinforcement Learning (MARL):** Each patrol car could be its own independent agent, learning to cooperate with other car-agents to achieve a common goal.

Conclusion

This project successfully demonstrates the potential of Q-learning for optimizing patrol dispatch operations. By introducing a holistic queue + locations state representation, we have fundamentally enhanced the agent's ability to perceive its environment. This allows it to learn a sophisticated and strategic policy that moves beyond simple, greedy decision-making. The developed framework is not only a practical application of reinforcement learning but also a scalable model for creating smarter, more efficient, and safer cities. 🚨